Creating a React application with Webpack

Step 1: Create a Project Folder

```
mkdir react-webpack-app
cd react-webpack-app
```

Step 2: Initialize npm

```
npm init -y
```

This creates:

```
package.json
```

Step 3: Install React

```
npm install react react-dom
```

Step 4: Install Development Dependencies

```
npm install --save-dev \
webpack \
webpack-cli \
```

```
webpack-dev-server \
@babel/core \
babel-loader \
@babel/preset-env \
@babel/preset-react \
html-webpack-plugin \
css-loader \
style-loader
```

Windows (single line):

```
npm install --save-dev webpack webpack-cli webpack-dev-server
@babel/core babel-loader @babel/preset-env @babel/preset-reac
t html-webpack-plugin css-loader style-loader
```

Step 5: Create the Folder Structure

```
react-webpack-app/
│
├── src/
│   ├── App.jsx
│   ├── index.jsx
│   └── styles.css
│
├── public/
│   └── index.html
│
├── webpack.config.js
├── babel.config.json
├── package.json
└── .gitignore
```

Step 6: Create `public/index.html`

```
<!DOCTYPE html>
<html>
<head>
    <meta charset="UTF-8">
    <title>React + Webpack</title>
</head>
<body>
    <div id="root"></div>
</body>
</html>
```

Step 7: Create `src/App.jsx`

```
export default function App() {
    return (
        <div>
            <h1>Hello React + Webpack!</h1>
        </div>
    );
}
```

Step 8: Create `src/index.jsx`

```
import React from "react";
import ReactDOM from "react-dom/client";
import App from "./App";
import "./styles.css";

const root = ReactDOM.createRoot(document.getElementById("roo
t"));
```

```
root.render(<App />);
```

Step 9: Create `src/styles.css`

```
body {
    font-family: Arial;
    background: #f5f5f5;
}

h1 {
    color: royalblue;
}
```

Step 10: Configure Babel

Create `babel.config.json`

```
{
  "presets": [
    "@babel/preset-env",
    "@babel/preset-react"
  ]
}
```

Step 11: Configure Webpack

Create `webpack.config.js`

```
const path = require("path");
const HtmlWebpackPlugin = require("html-webpack-plugin");
```

```
module.exports = {
    mode: "development",

    entry: "./src/index.jsx",

    output: {
        path: path.resolve(__dirname, "dist"),
        filename: "bundle.js",
        clean: true,
    },

    resolve: {
        extensions: [".js", ".jsx"],
    },

    module: {
        rules: [
            {
                test: /\.(js|jsx)$/,
                exclude: /node_modules/,
                use: "babel-loader",
            },
            {
                test: /\.css$/,
                use: ["style-loader", "css-loader"],
            },
        ],
    },

    plugins: [
        new HtmlWebpackPlugin({
            template: "./public/index.html",
        }),
    ],
```

```
    devServer: {
        static: "./dist",
        port: 3000,
        open: true,
        hot: true,
    },
};
```

Step 12: Update `package.json`

Add the scripts:

```
"scripts": {
  "start": "webpack serve --mode development",
  "build": "webpack --mode production"
}
```

Step 13: Start the Development Server

```
npm start
```

Open:

```
http://localhost:3000
```

You should see:

```
Hello React + Webpack!
```

Step 14: Create a Production Build

```
npm run build
```

Webpack generates:

```
dist/
├── bundle.js
└── index.html
```

This `dist` folder is ready to deploy to a static hosting service.

Complete Workflow

```
Create Project
      |
      ▼
npm init
      |
      ▼
Install React
      |
      ▼
Install Webpack
      |
      ▼
Install Babel
      |
      ▼
Create Source Files
      |
      ▼
Configure Babel
      |
      ▼
Configure Webpack
      |
      ▼
npm start
      |
      ▼
Development Server
      |
      ▼
npm run build
```

```
      |
      ▼
dist/
      |
      ▼
Deploy
```

How this compares to Vite

With Vite, the equivalent setup is dramatically simpler:

```
npm create vite@latest react-vite-app
cd react-vite-app
npm install
npm run dev
```

Vite generates the project structure and configuration for you, whereas with Webpack you're assembling the build pipeline yourself. That's why Webpack is often taught to explain the underlying mechanics of modern frontend tooling, while Vite is favored for day-to-day development.